

cc-agent-java 架构文档

这是一个 Java Spring Boot 学习项目。通过构建一个能调用 Claude API、读写文件、执行命令的 Agent，来学习 Java 和 Spring Boot。对你来说，Java/Spring Boot 是全新的，所以这篇文档用最直白的方式解释每个组件。

一、这个项目做什么

用户通过 HTTP 请求发一条消息（比如“帮我读 main.java”），程序收到后：

1. 把消息发给 Claude API
2. Claude 可能回复文本，也可能说“我需要读文件”（这叫 tool call）
3. 如果 Claude 要读文件，程序帮它读
4. 把文件内容发回给 Claude
5. Claude 看完内容后回复最终答案
6. 把答案返回给用户

用户：“帮我读 main.java”

→ 程序：发给 Claude

→ Claude：“我要调用 file_read 工具”

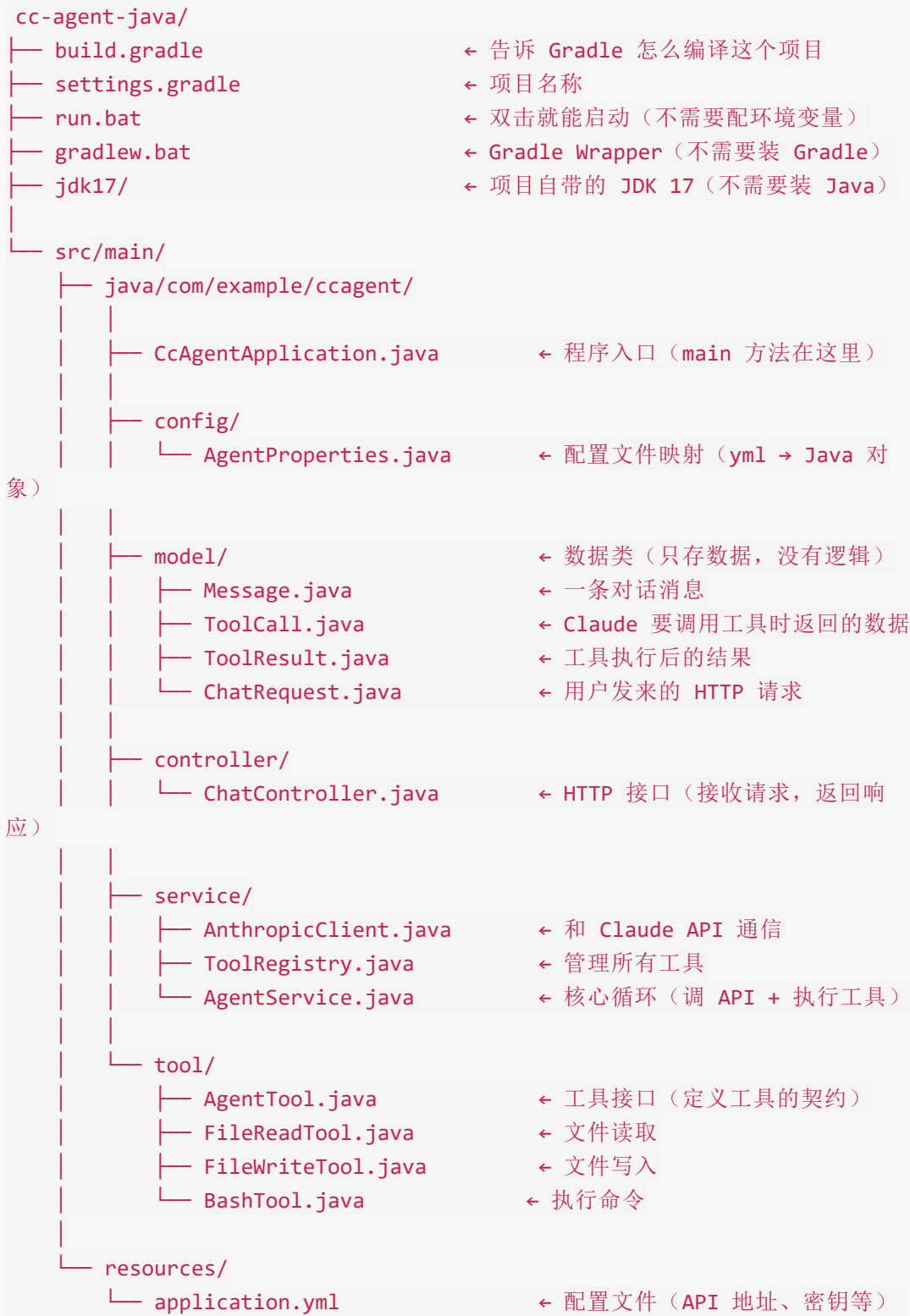
→ 程序：执行 file_read，读到文件内容

→ 程序：把文件内容发给 Claude

→ Claude：“文件内容是 XXX”

→ 程序：返回给用户

二、项目文件结构



三、逐组件解释

3.1 build.gradle — 项目怎么编译

```
plugins {
    id 'java' // 启用 Java 编译能力
    id 'org.springframework.boot' version '3.2.0' // Spring Boot 插件
    id 'io.spring.dependency-management' version '1.1.4' // 自动管理依赖版本
}

dependencies {
    implementation 'org.springframework.boot:spring-boot-starter-web'
    // spring-boot-starter-web 包含了：
    // - Spring MVC（处理 HTTP 请求）
    // - Jackson（JSON 和 Java 对象的互转）
    // - Tomcat（内嵌的 Web 服务器，监听 8080 端口）
}
```

白话解释： 这个文件告诉 Gradle“我需要这些库，帮我去网上下载”。Gradle 会自动从 Maven Central（类似 App Store）下载所有需要的依赖。

3.2 CcAgentApplication.java — 程序的入口

```
@SpringBootApplication
public class CcAgentApplication {
    public static void main(String[] args) {
        SpringApplication.run(CcAgentApplication.class, args);
    }
}
```

白话解释：

- `main` 方法是 Java 程序的入口（和 C++ 的 `int main()` 一样）
- `SpringApplication.run()` 启动 Spring Boot
- Spring Boot 启动后做了三件事：
- 扫描当前目录下所有标记了 `@Component` 的类，创建它们的实例
- 启动内嵌的 Tomcat Web 服务器（监听 8080 端口）
- 注册所有 HTTP 接口（`@GetMapping`、`@PostMapping` 等）

类比： 你在 C++ 里启动一个 RPC 服务时，需要手动写 main 函数、创建 server、注册 handler、开始监听。Spring Boot 把这些都做了，你只需要标记哪些类需要被管理。

3.3 AgentProperties.java — 配置文件映射

```
@ConfigurationProperties(prefix = "agent")
public record AgentProperties(
    String apiEndpoint,    // API 地址
    String apiKey,         // API 密钥
    String model,          // 模型名称
    int maxToolRounds     // 最大工具调用轮数
) {}
```

白话解释：

- application.yml 里写了 agent.api-endpoint: https://api.anthropic.com/...
- Spring 自动把 yml 里的值映射到这个 Java 对象的字段
- yml 用横杠命名 (api-endpoint)，Java 用驼峰命名 (apiEndpoint)，Spring 自动转换

类比： 你在 C++ 里读 JSON 配置文件，需要手动解析 JSON、赋值给 struct 字段。Spring 帮你自动做了这件事。

3.4 model/ — 数据类

Message.java — 一条对话消息：

```
public record Message(String role, String content) {}
// role = "user" (用户说的) 或 "assistant" (AI 说的)
// content = 消息内容
```

ToolCall.java — Claude 要调用工具时返回的数据：

```
public record ToolCall(String id, String name, Map<String, Object> input)
{}
// id = 这次调用的唯一 ID
// name = 工具名称 (如 "file_read")
// input = 工具参数 (如 {"path": "main.java"})
```

ToolResult.java — 工具执行后的结果：

```
public record ToolResult(String toolCallId, String content, boolean
isError) {}
// toolCallId = 关联到哪个 ToolCall
// content = 执行结果
// isError = 是否失败
```

ChatRequest.java — 用户发来的 HTTP 请求：

```
public record ChatRequest(String message) {}
// 用户发 POST /api/chat 时，请求体 JSON 自动转成这个对象
// {"message": "帮我看 main.java"} → ChatRequest("帮我看 main.java")
```

白话解释： Record 是 Java 16+ 的"不可变数据类"。你定义一个 Record，编译器自动帮你生成构造函数、getter 方法、equals、hashCode、toString。比 C++ 的 struct 方便——C++ 需要自己写 `operator==` 和 `hash`。

3.5 ChatController.java — HTTP 接口

```
@RestController                // 标记：这是一个 HTTP 控制器
@RequestMapping("/api")        // 所有接口都以 /api 开头
public class ChatController {

    @Autowired                  // 自动注入 AgentService 实例
    private AgentService agentService;

    @PostMapping("/chat")        // 处理 POST /api/chat
    public CompletableFuture<Map<String, String>> chat(
        @RequestBody ChatRequest request) {

        String userMessage = request.message();
        return agentService.run(userMessage)
            .thenApply(reply -> Map.of("reply", reply));
    }
}
```

白话解释（从请求到响应的完整流程）：

1. 用户用 curl 或 Postman 发 `POST http://localhost:8080/api/chat`
2. 请求体是 JSON： `{"message": "帮我看 main.java"}`

3. Tomcat 收到请求（它在 8080 端口监听）
4. Tomcat 把请求转发给 Spring
5. Spring 看到 `@PostMapping("/chat")`，知道要调用 `chat()` 方法
6. `@RequestBody` 把 JSON 请求体自动转成 `ChatRequest` 对象
7. `chat()` 方法调用 `agentService.run()`，传入用户消息
8. `agentService.run()` 是异步的（`@Async`），返回 `CompletableFuture`
9. `thenApply` 等 `run()` 完成后把结果转成 `{"reply": "..."} 格式`
10. Spring 把 Map 转成 JSON 返回给用户

类比：你在 C++ 里用 Thrift 定义了一个 RPC 接口，Thrift 自动生成 server handler 的骨架代码。Spring 的 `@PostMapping` 就是类似的东西——你定义“哪个 URL 调哪个方法”，框架自动处理 HTTP、JSON 解析。

3.6 AgentService.java — 核心循环

```

@Service
public class AgentService {

    @Autowired private AnthropicClient anthropicClient; // 和 Claude API
    通信
    @Autowired private ToolRegistry toolRegistry; // 管理所有工具

    @Async
    public CompletableFuture<String> run(String userMessage) {
        // 1. 创建对话历史
        List<Message> history = new ArrayList<>();
        history.add(new Message("user", userMessage));

        // 2. 循环（最多 10 轮）
        for (int round = 0; round < 10; round++) {
            // 3. 调 Claude API
            AnthropicResponse response = anthropicClient.chat(history);

            // 4. 如果没有工具调用，返回文本
            if (response.toolCalls().isEmpty()) {
                return CompletableFuture.completedFuture(response.text());
            }

            // 5. 执行每个工具
            for (ToolCall tc : response.toolCalls()) {
                ToolResult result = toolRegistry.execute(tc.name(),
                tc.input());
                history.add(new Message("user", "工具结果: " +
                result.content()));
            }
            return CompletableFuture.completedFuture("超过最大轮数");
        }
    }
}

```

白话解释（这个循环做什么）：

1. 把用户消息放入对话历史
2. 把历史发给 Claude API
3. Claude 回复了两种可能：
4. 纯文本 → 结束，返回给用户
5. 要调工具 → 执行工具，把结果加入历史，回到第 2 步
6. 如果 Claude 一直要调工具，最多循环 10 次

为什么 `@Async`：这个循环里调 Claude API 是慢的（要等网络响应）。如果不加 `@Async`，Tomcat 的请求线程会被阻塞，其他用户的请求要排队。加了 `@Async` 后，`AgentService.run()` 在另一个线程里跑，Tomcat 线程立即释放去处理下一个请求。

3.7 AnthropicClient.java — 调用 Claude API

```
@Component
public class AnthropicClient {

    @Autowired private AgentProperties properties; // 配置（API 地址、密
    钥、模型）

    public AnthropicResponse chat(List<Message> history) {
        // 1. 构建请求体 JSON
        Map<String, Object> body = new HashMap<>();
        body.put("model", properties.model());
        body.put("max_tokens", 4096);
        body.put("messages", history.stream()
            .map(m -> Map.of("role", m.role(), "content", m.content()))
            .toList());

        // 2. 发 HTTP POST 请求到 Claude API
        HttpRequest request = HttpRequest.newBuilder()
            .uri(URI.create(properties.apiEndpoint()))
            .header("x-api-key", properties.apiKey())
            .POST(HttpRequest.BodyPublishers.ofString(toJson(body)))
            .build();

        HttpResponse<String> response = httpClient.send(request, ...);

        // 3. 解析响应 JSON
        // 提取 text（纯文本）和 tool_use（工具调用）
        return new AnthropicResponse(text, toolCalls);
    }
}
```

白话解释：

- 这个类做的事情和你用 Python 调 OpenAI API 一样：构建 JSON 请求体 → 发 HTTP → 解析 JSON 响应
- 用 Java 内置的 `HttpClient`（不需要引入额外的 HTTP 库）
- 用 Jackson 做 JSON 和 Java 对象的互转

类比： 你在 C++ 里发 protobuf 请求到 RPC 服务端——你需要构建 protobuf 消息、序列化、发出去、收到响应后反序列化。 `AnthropicClient` 做的就是这样的事情，只是格式是 JSON 而不是 protobuf。

3.8 ToolRegistry.java — 管理所有工具

```
@Component
public class ToolRegistry {

    private final Map<String, AgentTool> tools;

    @Autowired
    public ToolRegistry(List<AgentTool> toolList) {
        // Spring 自动把所有实现了 AgentTool 接口的类的实例注入到这个 List
        // FileReadTool、FileWriteTool、BashTool 都会在里面
        this.tools = toolList.stream()
            .collect(Collectors.toMap(AgentTool::getName, t -> t));
    }

    public ToolResult execute(String name, Map<String, Object> input) {
        AgentTool tool = tools.get(name); // 按名字查找工具
        if (tool == null) return error("工具不存在");
        try {
            return new ToolResult(name, tool.execute(input), false);
        } catch (Exception e) {
            return new ToolResult(name, e.getMessage(), true);
        }
    }
}
```

白话解释：

- 构造函数参数是 `List<AgentTool> toolList`
- Spring 会自动找到所有实现了 `AgentTool` 接口的类（`FileReadTool`、`FileWriteTool`、`BashTool`），把它们的实例放到这个 List 里，传给构造函数
- 构造函数把这些工具放到一个 Map 里（key=工具名，value=工具实例）
- `execute()` 方法按名字查找工具并执行

类比： 你在 C++ 里可能有一个 `std::map<std::string, std::function<...>>` 来 dispatch 不同的 RPC 方法。 `ToolRegistry` 做的就是这件事——按名字找到对应的处理函数并调用。

3.9 AgentTool.java — 工具接口

```
public interface AgentTool {  
    String getName();           // 工具名称  
    String getDescription();    // 工具描述  
    String execute(Map<String, Object> input) throws Exception; // 执行  
}
```

白话解释:

- `interface` 在 Java 里类似于 C++ 的纯虚类——定义了一组方法签名，任何实现了这个接口的类都必须实现这些方法
- 所有工具（FileReadTool、FileWriteTool、BashTool）都实现这个接口
- 这样 ToolRegistry 不需要知道具体是哪个工具，只需要知道它实现了 AgentTool，就能调用它的 execute() 方法

3.10 FileReadTool.java — 文件读取

```
@Component  
public class FileReadTool implements AgentTool {  
    @Override public String getName() { return "file_read"; }  
    @Override public String getDescription() { return "读取文件内容"; }  
  
    @Override public String execute(Map<String, Object> input) throws  
Exception {  
        String path = (String) input.get("path");  
        return Files.readString(Path.of(path));  
    }  
}
```

白话解释:

- `@Component` — 告诉 Spring: "自动创建这个类的实例"
- `implements AgentTool` — 必须实现 getName、getDescription、execute 三个方法
- `@Override` — 标记这是实现接口的，如果方法名写错编译器会报错
- `execute()` — 从参数中取出文件路径，读取文件内容并返回

3.11 FileWriteTool.java — 文件写入

同 FileReadTool, 只是 `execute()` 里是 `Files.writeString()`。

3.12 BashTool.java — 执行命令

```
@Component
public class BashTool implements AgentTool {
    @Override public String getName() { return "bash"; }
    @Override public String getDescription() { return "执行 shell 命令"; }

    @Override public String execute(Map<String, Object> input) throws
Exception {
        String command = (String) input.get("command");
        // Runtime.getRuntime().exec() = 执行系统命令
        // 你在 C++ 里可能用过 system() 或 popen()
        // 这里用 Java 的等价版本
        Process process = Runtime.getRuntime().exec(
            new String[]{" /bin/sh", "-c", command});
        String output = new
String(process.getInputStream().readAllBytes());
        int exitCode = process.waitFor();
        if (exitCode != 0) {
            throw new RuntimeException("命令失败 (退出码 " + exitCode +
            ")");
        }
        return output;
    }
}
```

3.13 application.yml — 配置文件

```
agent:
  api-endpoint: https://api.anthropic.com/v1/messages
  api-key: ${ANTHROPIC_API_KEY}          # 从环境变量读取, 不写死密钥
  model: claude-sonnet-4-20250514
  max-tool-rounds: 10

server:
  port: 8080                             # Tomcat 监听这个端口
```

四、从请求到响应的完整链路

下面跟踪一个完整请求："帮我看 main.java"。

1. 用户发请求

```
POST http://localhost:8080/api/chat
Body: {"message": "帮我看 main.java"}
```

2. Tomcat 收到请求（它在 8080 端口监听）

→ 转发给 Spring

3. Spring 匹配路由

@PostMapping("/chat") → 调用 ChatController.chat()

4. ChatController.chat()

→ 从请求体取出 message
→ 调用 agentService.run("帮我看 main.java")
→ 返回 CompletableFuture（异步）

5. AgentService.run() [在独立线程执行]

→ 创建 history = [Message("user", "帮我看 main.java")]
→ 第 1 轮：
→ anthropicClient.chat(history)
→ 发 HTTP POST 到 Claude API
→ Claude 回复: tool_use(name="file_read", input={path: "main.java"})
→ toolCalls 不为空
→ toolRegistry.execute("file_read", {path: "main.java"})
→ FileReadTool 读取文件，返回内容
→ history 加入工具结果
→ 第 2 轮：
→ anthropicClient.chat(history) // 这次 history 包含了文件内容
→ Claude 回复: "文件内容是 public class Main { ... }"
→ toolCalls 为空
→ 返回 "文件内容是 public class Main { ... }"

6. ChatController.thenApply()

→ 收到结果
→ 转成 {"reply": "文件内容是..."}

7. 返回给用户

五、Spring Boot 的关键概念

概念	白话解释	C++ 类比
<code>@Component</code>	标记"这个类需要 Spring 自动创建实例"	告诉工厂"这个东西需要生产"
<code>@Service</code>	和 <code>@Component</code> 一样，语义上表示"业务逻辑类"	同上
<code>@RestController</code>	和 <code>@Component</code> 一样，语义上表示"HTTP 控制器"	同上
<code>@Autowired</code>	把 Spring 容器里的实例连接到这个字段（传引用）	工厂把生产好的零件装配到这个位置
<code>@PostMapping("/chat")</code>	当用户 POST 请求到 <code>/api/chat</code> 时，调用这个方法	注册 RPC 路由
<code>@RequestBody</code>	把 HTTP 请求体 JSON 自动转成 Java 对象	JSON 反序列化
<code>@Async</code>	这个方法在独立线程执行，不阻塞调用者	<code>std::async</code>
Record	不可变数据类，编译器自动生成 <code>getter/equals/hashCode</code>	<code>struct</code> + 自动生成运算符
<code>List<AgentTool></code> 作为参数	Spring 自动把所有 <code>AgentTool</code> 实现类的实例注入	自动收集所有 handler

六、为什么这样设计

6.1 为什么 Agent 循环是 for 而不是 while

用 `for (int round = 0; round < 10; round++)` 而不是 `while(true)`，因为： - 防止无限循环（如果 Claude 一直要调工具） - 10 是安全上限

6.2 为什么 AgentService 要 @Async

这个循环里调 Claude API 可能慢（网络延迟、大文件处理）。如果不加 `@Async`： - Tomcat 线程会被阻塞 - 其他用户的请求要排队等待 - 多个用户同时使用时体验很差

加了 `@Async` 后，`AgentService.run()` 在独立线程跑，Tomcat 线程立即释放。

6.3 为什么工具用接口而不是直接写死在循环里

- 接口让新增工具很简单：新建一个类，实现 AgentTool，加 @Component，完事
- 不需要修改 AgentService 或 ToolRegistry
- 这是"开闭原则"——对扩展开放，对修改关闭

七、从 C++ 到 Java 的关键思维转换

C++ 习惯	Java 做法
手动 new 对象，手动 delete	Spring 自动创建和管理，你不需要关心生命周期
手写函数指针表 dispatch RPC	Spring 自动收集实现了接口的类
手写 JSON 解析	Jackson 自动转成 Java 对象
手写 main 启动服务器	SpringApplication.run() 自动启动 Tomcat
头文件 <code>.h</code> + 实现文件 <code>.cpp</code>	一个 <code>.java</code> 文件搞定（没有头文件的概念）
<code>make</code> 或 <code>CMake</code> 编译	<code>gradlew build</code> 自动下载依赖并编译
<code>.exe</code> 运行	<code>java -jar xxx.jar</code> 在 JVM 里运行